JadiSan — Comprehensive Application Documentation & File-by-File Reference (A to Z)

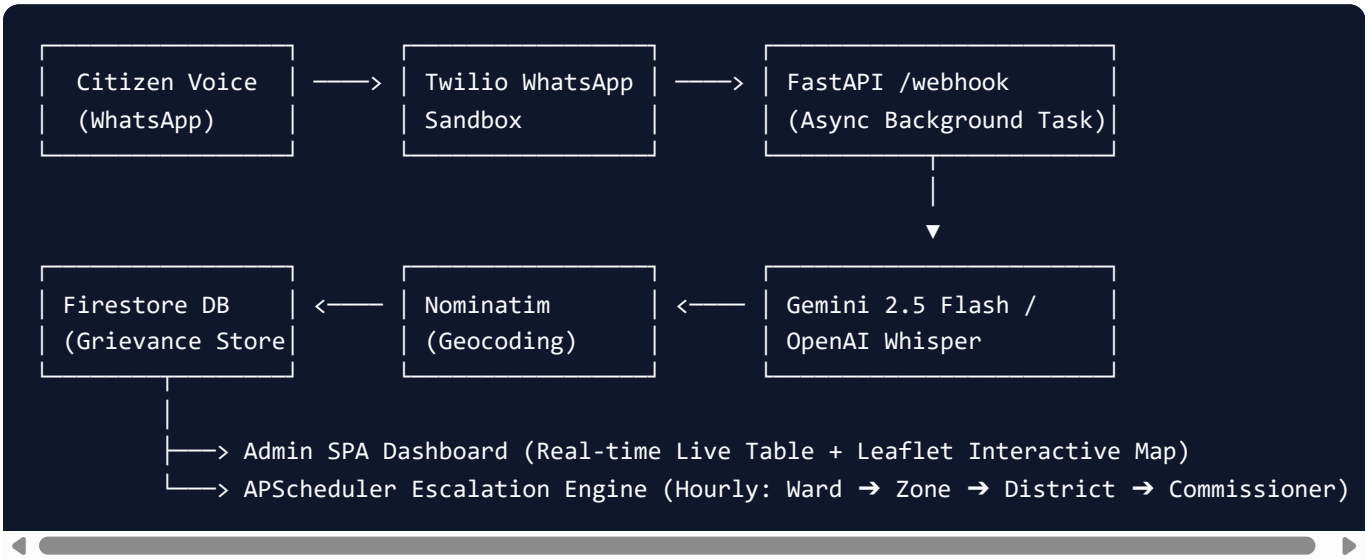
Application Name: JadiSan (Voice-Based Civic Grievance Reporting System)
Developers: Team ByteCraft, Panimalar Engineering College
Target Platform: WhatsApp (via Twilio Sandbox), FastAPI Backend, Google Cloud Firestore, Admin SPA Dashboard

---

1. Executive Summary & Architecture Overview

JadiSan is an intelligent, voice-first civic grievance reporting platform designed to eliminate technological and linguistic barriers for citizens reporting municipal issues (e.g., potholes, water leaks, garbage overflow, broken streetlights, sewage backup). Citizens simply send a voice note in their native language (e.g., Tamil, Hindi, English) via WhatsApp.

System Architecture Flow



End-to-End Pipeline

1. **Voice Input Ingestion:** Twilio webhook receives voice message (OGG/WAV) or text.
2. **Local AI Transcription:** OpenAI Whisper model (`small` / `base`) transcribes multilingual audio locally without cloud API costs or privacy leaks.
3. **LLM Classification & Duplicate Detection:** Google Gemini 2.5 Flash classifies category and severity, extracts location text, generates a single-sentence English summary, and compares against recent Firestore entries for semantic duplicate detection.

4. **Geocoding:** Nominatim (OpenStreetMap) converts text locations into precise latitude and longitude coordinates (with fallback to native WhatsApp GPS pins).
5. **Database & Escalation:** Firestore logs the structured grievance. If unresolved, APScheduler automatically escalates the grievance through official tiers (`Ward` ➔ `Zone` ➔ `District` ➔ `Commissioner`).
6. **Automated Notification:** Twilio sends a formatted WhatsApp confirmation back to the citizen with a tracking Reference ID (e.g., `JDS-20260904-0001`).

---

2. Comprehensive A to Z File Index & Reference

The codebase consists of **20 files** across root, backend, and dashboard modules. Below is the exhaustive reference listed in **strict alphabetical order from A to Z**.

File 1: `.env.example`

- **Location:** `file:///c:/Projects/jadisan/.env.example`
- **Component:** Environment Configuration Template
- **Purpose:** Serves as the blueprint for environment variable initialization across deployment environments.
- **Key Variables:**
- `TWILIO_ACCOUNT_SID` : Twilio API Account Identifier.
- `TWILIO_AUTH_TOKEN` : Secret auth token for Twilio REST client authentication.
- `TWILIO_WHATSAPP_NUMBER` : Designated WhatsApp sender number (`whatsapp:+14155238886`).
- `GEMINI_API_KEY` : API key for Google Generative AI (Gemini 2.5 Flash).
- `FIREBASE_CREDENTIALS_PATH` : Path to Firebase Service Account JSON file.
- `WHISPER_MODEL` : Local Whisper model size (`tiny` , `base` , `small` , `medium` , `large`). Default is `small` .
- `ESCALATION_HOURS` : SLA duration (in hours) before auto-escalation triggers (default: `48`).
- **Interactions:** Loaded by `backend/main.py` via `python-dotenv` before module initializations.

File 2: `README.md`

- **Location:** `file:///c:/Projects/jadisan/README.md`
- **Component:** Core Documentation & Onboarding Manual
- **Purpose:** Provides comprehensive technical documentation, architecture diagrams, prerequisite tables, setup steps, ngrok webhook configuration guidelines, Firestore schema definitions, API endpoint catalogs, and testing instructions.
- **Key Highlights:**
- Describes testing pipeline commands using `curl` .
- Details local development execution instructions via `uvicorn main:app --reload` .

File 3: `backend/analyzer.py`

- **Location:** `file:///c:/Projects/jadisan/backend/analyzer.py`
- **Component:** Artificial Intelligence & Natural Language Processing Engine
- **Purpose:** Leverages Google Gemini 2.5 Flash (`gemini-3.6-flash` API endpoint) to perform intent classification, severity evaluation, location extraction, language detection, and duplicate grievance identification.
- **Key Definitions:**
- Constants: `CATEGORIES` (`Pothole` , `Water Leak` , `Garbage` , `Broken Streetlight` , `Sewage` , `Road Damage` , `Other`), `SEVERITIES` (`Low` , `Medium` , `High` , `Critical`), `TIERS` (`Ward` , `Zone` , `District` , `Commissioner`).
- `init_gemini()` : Configures `google.generativeai` with the `GEMINI_API_KEY` .
- `_SYSTEM_PROMPT` : Directs the model to output strict JSON schema without markdown block code fences.
- `analyze_complaint(transcript, recent_complaints)` : Formats context from up to 50 recent complaints, invokes `generate_content_async` , strips potential code fences, parses JSON, and enforces schema sanitization fallback.
- **Dependencies:** `google.generativeai` , `json` , `re` , `logging` .

File 4: `backend/escalator.py`

- **Location:** `file:///c:/Projects/jadisan/backend/escalator.py`
- **Component:** SLA & Tier Escalation Engine
- **Purpose:** Manages background time-based grievance escalation schedules and provides manual single/bulk escalation handlers for municipal administrative tiers.
- **Key Definitions:**
- Constant: `TIER_LADDER = ["Ward", "Zone", "District", "Commissioner"]` .
- `next_tier(current_tier)` : Returns the next administrative tier or `None` if at Commissioner.
- `run_escalation_check()` : Queries Firestore for open grievances exceeding `ESCALATION_HOURS` and promotes them up the tier ladder.
- `escalate_single(complaint_id)` : Manually promotes a specific grievance by document ID.
- `start_scheduler()` & `stop_scheduler()` : Lifecycle hooks managing `AsyncIOScheduler` executing hourly cron checks.
- **Dependencies:** `apscheduler.schedulers.asyncio.AsyncIOScheduler` , `firestore_client` .

File 5: `backend/firestore_client.py`

- **Location:** `file:///c:/Projects/jadisan/backend/firestore_client.py`
- **Component:** Database Data Access Layer (DAO)
- **Purpose:** Encapsulates all read and write interactions with Google Cloud Firestore, including reference ID generation, query filtering, and document updates.
- **Key Definitions:**

- `init_firestore()` : Initializes `firebase_admin` certificate credentials.
- `_generate_reference_id()` : Generates sequential daily reference IDs in format `JDS-YYYYMMDD-XXXX` .
- `get_recent_complaints(limit)` : Retrieves latest grievances for duplicate contextual comparison.
- `get_all_complaints(...)` : Supports multi-parameter filtering (`category` , `severity` , `status` , `tier`).
- `create_complaint(data)` : Persists new grievance documents into the `complaints` collection.
- `increment_affected_count(complaint_id)` : Atomically increments complaint impact count when duplicates are detected.
- `escalate_complaint(complaint_id, new_tier)` : Updates tier status and timestamp.
- **Dependencies:** `firebase_admin` , `google.cloud.firestore` .

File 6: `backend/geocoder.py`

- **Location:** `file:///c:/Projects/jadisan/backend/geocoder.py`
- **Component:** Geographical Coordinates Resolver
- **Purpose:** Converts free-text location descriptions into exact geographic coordinates `(latitude, longitude)` using OpenStreetMap's Nominatim API.
- **Key Definitions:**
- `_geocoder` : Instantiated `Nominatim(user_agent="jadisan_civic_app")` .
- `_cache` : In-memory dictionary store preventing redundant API requests.
- `_lock` : `asyncio.Lock()` enforcing Nominatim's strict 1 request/second Terms-of-Service rate limit.
- `geocode_location(location_text)` : Performs rate-limited, country-restricted (`country_codes="in"`) geocoding wrapped in `asyncio.to_thread` .
- **Dependencies:** `geopy.geocoders.Nominatim` , `asyncio` .

File 7: `backend/jadisan-2-firebase-adminsdk-fbsvc-a6efc4cd0e.json`

- **Location:** `file:///c:/Projects/jadisan/backend/jadisan-2-firebase-adminsdk-fbsvc-a6efc4cd0e.json`
- **Component:** Firebase Credentials Service Key
- **Purpose:** Service Account JSON configuration containing private keys, client email, project ID, and authorization URIs required by `firebase-admin` SDK for authenticating against Google Cloud Project `jadisan-2` .
- **Security Note:** Confidential authentication file. Must be protected and excluded from version control in production setups.

File 8: `backend/main.py`

- **Location:** `file:///c:/Projects/jadisan/backend/main.py`
- **Component:** Application Core & FastAPI Web Webhook Controller

- **Purpose:** Serves as the primary entry point for the REST API and Twilio Webhook server. Orchestrates async background processing of voice notes, audio downloading, transcription, analysis, geocoding, database persistence, and WhatsApp response dispatching.
- **Key Routes & Functions:**
- `lifespan(app)` : Initializes Firestore, Gemini, Whisper model caching, and starts APScheduler on application startup.
- `POST /webhook` : Handles incoming WhatsApp payloads from Twilio, returns instant XML `<Response/>` to avoid webhook timeouts, and offloads processing to `_process_complaint` .
- `_process_complaint(...)` : Asynchronous worker downloading audio via Basic Auth, running local Whisper transcription, querying Gemini, geocoding, updating Firestore, and triggering Twilio messaging.
- `GET /api/complaints` : Admin API endpoint serving filtered complaint lists.
- `POST /api/escalate/all` & `POST /api/escalate/{id}` : Manual escalation endpoints.
- `GET /` : Serves the static single-page admin dashboard HTML interface.
- **Dependencies:** `fastapi` , `httpx` , `tempfile` , `pathlib` , `analyzer` , `escalator` , `firestore_client` , `geocoder` , `transcriber` , `twilio_client` .

File 9: `backend/make_test_audio.ps1`

- **Location:** `file:///c:/Projects/jadisan/backend/make_test_audio.ps1`
- **Component:** Synthetic Test Audio Generator Script
- **Purpose:** Windows PowerShell script using `.NET System.Speech.Synthesis` engine to generate local synthetic WAV audio files (`test_complaint.wav`) containing spoken test grievance text for offline transcription verification.
- **Commands:** Creates `SpeechSynthesizer` object, synthesizes spoken grievance text, and writes output to disk.

File 10: `backend/quick_test.py`

- **Location:** `file:///c:/Projects/jadisan/backend/quick_test.py`
- **Component:** Quick Local Verification Utility
- **Purpose:** Minimal test runner script used to verify that the local Whisper model can load correctly and transcribe `test_complaint.wav` synchronously without spinning up the full FastAPI application stack.

File 11: `backend/requirements.txt`

- **Location:** `file:///c:/Projects/jadisan/backend/requirements.txt`
- **Component:** Python Package Dependency Specification
- **Purpose:** Enumerates exact version requirements for all runtime backend dependencies.
- **Listed Packages:**
- `fastapi==0.115.5`

- `uvicorn[standard]==0.32.1`
- `python-multipart==0.0.17`
- `twilio==9.3.7`
- `openai-whisper==20250625`
- `ffmpeg-python==0.2.0`
- `google-generativeai==0.8.3`
- `firebase-admin==6.6.0`
- `apscheduler==3.10.4`
- `python-dotenv==1.0.1`
- `httpx==0.27.2`
- `geopy==2.4.1`
- `aiofiles==24.1.0`

File 12: `backend/test_complaint.ogg`

- **Location:** `file:///c:/Projects/jadisan/backend/test_complaint.ogg`
- **Component:** Sample Audio Binary File (Opus/OGG format)
- **Purpose:** Represents a sample audio file matching Twilio WhatsApp voice message encodings for testing audio download, temporary file storage, and Whisper transcription pipelines.

File 13: `backend/test_complaint.wav`

- **Location:** `file:///c:/Projects/jadisan/backend/test_complaint.wav`
- **Component:** Sample Audio Binary File (PCM/WAV format)
- **Purpose:** High-fidelity uncompressed synthesized audio sample generated by `make_test_audio.ps1` for local Whisper accuracy testing.

File 14: `backend/test_pipeline.py`

- **Location:** `file:///c:/Projects/jadisan/backend/test_pipeline.py`
- **Component:** Automated Unit Testing Suite
- **Purpose:** Contains unit tests covering individual backend components using `unittest` and `unittest.mock`.
- **Test Classes:**
- `TestEscalatorTierLadder`: Verifies transitions between `Ward` ➔ `Zone` ➔ `District` ➔ `Commissioner`.
- `TestGeminiAnalyzer`: Mocks Gemini API outputs, testing valid JSON parsing, markdown fence stripping, and fallback mechanisms for invalid categories/severities.
- `TestGeocoder`: Validates input validation, empty string handling, and cache hit efficiency.
- `TestTwilioClient`: Validates message body formatting for new complaints and duplicate issue alerts without invoking external REST calls.

File 15: `backend/test_server_mock.py`

- **Location:** `file:///c:/Projects/jadisan/backend/test_server_mock.py`
- **Component:** Full End-to-End Integration Test Suite
- **Purpose:** Launches the FastAPI application using HTTPX `AsyncClient` and `ASGITransport` with all external services (Firestore, Gemini, Twilio, Whisper) fully mocked.
- **Key Test Cases:**
- `test_webhook_returns_twiml` : Asserts XML HTTP 200 response.
- `test_health_endpoint` : Confirms `/api/health` returns status `ok` .
- `test_complaints_api_returns_list` : Verifies API JSON structure.
- `test_escalate_all_endpoint` & `test_escalate_single_not_found` : Asserts manual escalation behaviors.

File 16: `backend/transcriber.py`

- **Location:** `file:///c:/Projects/jadisan/backend/transcriber.py`
- **Component:** Offline Speech-to-Text Transcription Engine
- **Purpose:** Wraps local OpenAI Whisper model execution, ensuring model weights are loaded into RAM once at startup and reused across requests.
- **Key Definitions:**
- `load_model(model_size)` : Module-level singleton loader caching `whisper.Whisper` instance.
- `get_model()` : Safe getter enforcing model pre-loading checks.
- `transcribe_audio(audio_path)` : Async function delegating `model.transcribe()` to a worker thread via `asyncio.to_thread(..., fp16=False)` to prevent event loop freezing on CPU execution.
- **Dependencies:** `whisper` , `asyncio` , `pathlib` .

File 17: `backend/twilio_client.py`

- **Location:** `file:///c:/Projects/jadisan/backend/twilio_client.py`
- **Component:** WhatsApp Outbound Messaging Client
- **Purpose:** Constructs rich, formatted WhatsApp confirmation messages featuring category/severity emoji icons and dispatches them to citizens via Twilio REST API.
- **Key Functions:**
- `_get_client()` : Initializes `twilio.rest.Client` using environment credentials.
- `send_complaint_confirmation(...)` : Formats distinct WhatsApp messages for standard new complaints vs. duplicate issue notifications (which report updated citizen impact counts).
- **Dependencies:** `twilio.rest.Client` .

File 18: `dashboard/app.js`

- **Location:** `file:///c:/Projects/jadisan/dashboard/app.js`
- **Component:** Frontend Single-Page Application Logic
- **Purpose:** Handles administrative UI operations including data fetching from `/api/complaints`, live dashboard stats calculations, table sorting/filtering, manual single/bulk escalation requests, toast notifications, and Leaflet interactive map rendering.
- **Key Functions:**
- `loadComplaints()`: Fetches JSON feed and triggers view updates.
- `startAutoRefresh()`: Sets background polling interval (every 30 seconds).
- `updateStats()`: Dynamically calculates metrics (Total, Open, Critical, Today, Escalated).
- `applyFilters()` & `sortComplaints()`: Multi-column client-side table sorting and filter matching.
- `initMap()` & `updateMap()`: Renders interactive map pins colored by severity with divIcon styling.
- `escalateSingle(id)` & `escalateAll()`: Issues POST requests to escalation endpoints with UI feedback.
- **Dependencies:** Browser DOM API, Leaflet JavaScript Library (`L.map`).

File 19: `dashboard/index.html`

- **Location:** `file:///c:/Projects/jadisan/dashboard/index.html`
- **Component:** Frontend Admin Interface HTML Layout
- **Purpose:** Semantic HTML5 document defining the UI layout for the administrative control panel.
- **Key Sections:**
- `<header>`: Brand identity, pulse animation auto-refresh indicator, manual refresh button.
- `.stats-bar`: Metric summary cards.
- `.controls-bar`: Tab view switchers (Table View vs. Map View) and filter dropdown selects.
- `#viewTable`: Data table listing complaints with sortable column headers.
- `#viewMap`: Leaflet map container and color-coded severity legend.
- `#toastContainer`: Dynamic notification container.
- **External Resources:** Google Fonts (`Inter`, `JetBrains Mono`), Leaflet CSS/JS CDN (`1.9.4`).

File 20: `dashboard/styles.css`

- **Location:** `file:///c:/Projects/jadisan/dashboard/styles.css`
- **Component:** Modern Dark-Theme Design System
- **Purpose:** Provides custom CSS styling featuring dark navy theme `#0f172a`, teal accents `#14b8a6`, glassmorphism backdrop blur filters, custom metric card shadows, status badges, Leaflet popup customizations, responsive layouts, and toast animations.
- **Key Tokens:**
- `--bg-base: #0f172a`, `--bg-surface: #1e293b`, `--accent: #14b8a6`.
- Severity Badges: `--sev-low: #22c55e`, `--sev-medium: #eab308`, `--sev-high: #f97316`, `--sev-critical: #ef4444`.

- Tier Colors: `--tier-ward: #60a5fa`, `--tier-zone: #a78bfa`, `--tier-district: #fb923c`, `--tier-commissioner: #f43f5e`.

---

3. Data Schema & Integration Reference

Firestore Collection: `complaints`

```
{
  "reference_id": "JDS-20260904-0001",
  "sender_phone": "whatsapp:+919876543210",
  "transcript": "அண்ணா நகர் மெயின் ரோட்டில் பெரிய சாக்கடை அடைப்பு ஏற்பட்டு கழிவு
  "summary": "Major sewage blockage on Anna Nagar Main Road causing overflow on the street.",
  "category": "Sewage",
  "severity": "High",
  "location_text": "Anna Nagar Main Road",
  "latitude": 13.0850,
  "longitude": 80.2100,
  "status": "open",
  "tier": "Ward",
  "is_duplicate": false,
  "duplicate_of_id": null,
  "affected_count": 1,
  "language_detected": "Tamil",
  "created_at": "2026-09-04T12:00:00Z",
  "updated_at": "2026-09-04T12:00:00Z"
}
```

---

4. Setup, Deployment & Execution Guide

Prerequisites

- Python 3.11+
- FFmpeg (installed and added to system PATH for audio decoding)
- Firebase Cloud Firestore account
- Twilio WhatsApp Sandbox credentials

Installation Steps

```
# 1. Clone repository & change directory
cd c:/Projects/jadisan

# 2. Setup virtual environment & dependencies
cd backend
python -m venv venv
venv\Scripts\activate
pip install -r requirements.txt

# 3. Configure environment variables
cp ../.env.example .env
# Edit .env with your real credentials (API keys, serviceAccountKey.json)

# 4. Launch FastAPI development server
uvicorn main:app --reload --port 8000

# 5. Access Admin Dashboard in Browser
# http://localhost:8000/
```

Documentation compiled for JadiSan Application — Team ByteCraft, Panimalar Engineering College.